

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai – 602105

A PROJECT REPORT
ON
“STUDENT PERFORMANCE RISK PREDICTOR”

Web-Based Academic Risk Prediction Using k-Nearest Neighbors (KNN)

Submitted in partial fulfilment for the course
DSA0603 – DATA HANDLING AND VISUALIZATION

Submitted by
N. Harshavardhan (192324189)

Dhanajeyan j (192324211)

Siddarth P (192324264)

Under the Guidance of
Dr. S. Senthilvadivu

B.E. Artificial Intelligence and Data Science

SIMATS ENGINEERING

September 2026

DECLARATION

I, N. Harshavardhan (192324189), a student of B.E. Artificial Intelligence and Data Science at SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, hereby declare that the project entitled “Student Performance Risk Predictor: Web-Based Academic Risk Prediction Using k-Nearest Neighbors (KNN)” is my original work carried out as part of the requirements for the course DSA0603 – Data Handling and Visualization.

The project has been developed with the objective of applying data handling, visualization, preprocessing, and machine learning concepts to an educational problem. The system uses attendance, past academic grades, and study hours to classify students into “At Risk” or “Safe” categories using the k-Nearest Neighbors (KNN) algorithm.

I further declare that the work presented in this report has been completed under the guidance of Dr. S. Senthilvadivu and that the information and results included are presented for academic purposes.

Place: Chennai

Date: September 2026

Signature of the Student

N. Harshavardhan (192324189)

BONAFIDE CERTIFICATE

This is to certify that the project report entitled “Student Performance Risk Predictor: Web-Based Academic Risk Prediction Using k-Nearest Neighbors (KNN)” has been carried out by N. Harshavardhan (192324189), student of B.E. Artificial Intelligence and Data Science, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, under the guidance of Dr. S. Senthilvadivu, and is submitted in partial fulfilment of the requirements for the course DSA0603 – Data Handling and Visualization.

The work presented in this report is based on the design and development of a web application that processes academic indicators and predicts whether a student is likely to be classified as At Risk or Safe.

This project work is approved for academic submission and evaluation.

SIGNATURE

Guide

Dr. S. Senthilvadivu

Program / Department

Artificial Intelligence and Data Science

SIMATS Engineering

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who supported and guided me during the development of my project titled “Student Performance Risk Predictor: Web-Based Academic Risk Prediction Using k-Nearest Neighbors (KNN)”.

First and foremost, I express my heartfelt thanks to my project guide, Dr. S. Senthilvadivu, for providing valuable guidance, continuous encouragement, constructive suggestions, and academic support throughout the project. Her guidance helped me understand the practical application of data handling, visualization, and machine learning techniques.

I am grateful to SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, for providing the academic environment, laboratory facilities, learning resources, and support required to complete this work.

I also thank the faculty members of the Artificial Intelligence and Data Science department for their valuable inputs and for strengthening my understanding of data science and machine learning concepts.

Finally, I express my sincere thanks to my family and friends for their encouragement, motivation, and support throughout the completion of this project.

Signature of the Student

N. Harshavardhan (192324189)

ABSTRACT

Students who experience academic difficulties are often identified only after their performance has already declined. Early identification of academic risk can support timely guidance and appropriate intervention. The Student Performance Risk Predictor is a web-based application designed to classify students into “At Risk” or “Safe” categories using key academic indicators such as attendance percentage, past grades, and study hours.

The proposed system applies the k-Nearest Neighbors (KNN) classification algorithm. The input data is cleaned, transformed into a suitable numerical form, standardized where necessary, and divided into training and testing sets. For a new student, KNN calculates the distance between the student record and existing training records, identifies the nearest observations, and determines the predicted class from the neighboring labels.

The web application provides a simple interface for entering student information and displaying the predicted risk category. It also supports data visualization to help users understand patterns between attendance, grades, study hours, and academic risk. Evaluation can be performed using accuracy, precision, recall, F1-score, and confusion matrix results.

The project demonstrates how data handling, exploratory visualization, and machine learning can be integrated into a practical educational decision-support application. The system is intended as an academic support tool and not as a replacement for teacher or institutional judgment.

Keywords: Student Performance, Academic Risk, KNN, Machine Learning, Data Visualization, Attendance, Grades, Study Hours

TABLE OF CONTENTS

S.NO	CONTENT	PAGE
1	Introduction	1
1.1	Background and Context	1
1.2	Problem Statement	2
1.3	Objectives	2
1.4	Scope and Limitations	3
1.5	Organization of Report	3
2	Literature Review	4
2.1	Existing Solutions and Technologies	4
2.2	Comparative Analysis	5
2.3	Research Gap	6
2.4	Theoretical Framework	6
3	Methodology	7
3.1	System Architecture	7
3.2	Hardware and Software Requirements	8
3.3	Dataset and Data Preparation	9
3.4	kNN Model Design	10
3.5	Web Application Design	10
3.6	Implementation Process	11
4	Results and Discussion	12
4.1	Testing Methodology	12
4.2	Performance Metrics	13
4.3	Visualization and Prediction Analysis	14
4.4	Discussion	15
5	Reflection on Learning	16
5.1	Technical Skills Acquired	16
5.2	Data Handling and Visualization Skills	17
5.3	Problem Solving and Communication	17
5.4	Application of Engineering Practices	18
5.5	Personal Development	18
6	Conclusion	19
6.1	Summary of Achievements	19
6.2	Key Findings	20
6.3	Future Scope	20

6.4	Recommendations	21
7	References	22

CHAPTER 1: INTRODUCTION

1.1 Background and Context

Educational institutions collect large amounts of academic information, including attendance records, examination marks, internal assessment scores, and study-related information. When these data sources are analyzed systematically, they can provide useful indications about student performance and help educators identify students who may need additional academic support.

Traditional approaches to identifying struggling students often depend on manual observation, periodic examinations, or teacher experience. Although these methods remain important, they may not always provide an early and consistent indication of risk. A data-driven approach can complement human judgment by analyzing multiple student indicators together.

The Student Performance Risk Predictor focuses on three understandable and commonly available attributes: attendance percentage, past grades, and study hours. These features are used as input variables for a k-Nearest Neighbors classifier. The resulting prediction is presented through a web application so that the model can be used through a simple user interface.

The project is aligned with the DSA0603 – Data Handling and Visualization course because it combines data collection, preprocessing, exploratory analysis, visualization, machine learning classification, and presentation of results.

1.2 Problem Statement

Students at academic risk are often identified too late for effective intervention. The proposed system uses attendance, past grades, and study hours with the KNN algorithm to classify students as “At Risk” or “Safe”, helping educators identify potential academic risk earlier and supporting timely academic guidance.

1.3 Objectives

- To collect and organize student academic data containing attendance, past grades, study hours, and a risk label.
- To clean and preprocess the dataset so that it can be used effectively by a machine learning model.
- To explore relationships between academic indicators using suitable data visualizations.
- To implement the k-Nearest Neighbors (KNN) algorithm for binary classification.
- To evaluate the classifier using standard performance metrics such as accuracy, precision, recall, F1-score, and confusion matrix.
- To develop a user-friendly web interface for entering student details and viewing the predicted risk category.
- To provide a practical academic decision-support prototype while keeping human judgment central to intervention decisions.

1.4 Scope and Limitations

The scope of the project includes preprocessing academic records, training a KNN classifier, evaluating model performance, creating visualizations, and integrating the trained model into a web application. The application can be used with institutional records or a suitable public student-performance dataset such as the UCI Student Performance dataset, provided the required variables are available or derived appropriately.

The system is limited by the quality, representativeness, and completeness of the training data. Attendance, grades, and study hours alone cannot capture every factor affecting academic performance. Personal circumstances, teaching quality, subject difficulty, learning disabilities, financial challenges, and other contextual factors may influence student outcomes but are outside the core scope of this prototype.

The prediction should therefore be interpreted as a risk indicator rather than a definitive judgment about a student's ability or future performance.

1.5 Organization of Report

Chapter 1 introduces the project background, problem statement, objectives, scope, and report organization. Chapter 2 reviews existing approaches and establishes the research gap and theoretical foundation. Chapter 3 presents the dataset, preprocessing pipeline, KNN model, system architecture, requirements, and implementation process. Chapter 4 describes testing, performance evaluation, visualization, and discussion. Chapter 5 reflects on the technical and professional learning gained through the project. Chapter 6 summarizes the achievements and presents future scope and recommendations. Chapter 7 lists the references used for the project.

CHAPTER 2: LITERATURE REVIEW

2.1 Existing Solutions and Technologies

Student performance prediction has been studied using statistical analysis and machine learning methods. Common classification approaches include decision trees, logistic regression, support vector machines, kNN, random forests, and neural networks. These methods use historical academic data to discover patterns associated with performance outcomes.

Educational dashboards are another important technology for presenting student information. Visualizations such as bar charts, scatter plots, histograms, and correlation matrices can help users understand academic trends more quickly than raw tables.

The proposed system combines a lightweight machine learning method with a web-based interface. KNN is particularly suitable for a prototype because its basic idea is intuitive: similar student records are expected to have similar class labels.

2.2 Comparative Analysis

Traditional manual identification is highly dependent on teacher experience and periodic observation. Rule-based systems can provide simple alerts but may depend on fixed thresholds. More complex machine learning models may achieve strong predictive performance but can require additional tuning, computational resources, or explanation mechanisms.

Approach	Strength	Limitation	Role in Project
Manual monitoring	Uses teacher expertise	Time-consuming and subjective	Reference approach
Rule-based	Simple to implement	Fixed thresholds may miss patterns	Possible baseline
Decision tree	Easy to interpret	Can overfit without control	Alternative model
kNN	Simple, intuitive, non-parametric	Sensitive to scaling and k choice	Primary classifier
Web dashboard	Easy access and visualization	Depends on data quality	User interface

For this academic project, kNN provides a clear connection between similarity, distance calculation, and classification. Standardization is important because attendance, grades, and study hours may be measured on different scales.

2.3 Research Gap

A practical gap exists between machine learning experiments performed in notebooks and usable interfaces that present predictions in a form understandable to non-technical users. A model may demonstrate classification performance but still require technical knowledge to use.

The proposed project addresses this gap by combining data preprocessing, visualization, KNN classification, and a web interface. The system is designed to make the prediction workflow transparent: the user enters the academic indicators, the application processes the values, the trained classifier predicts the category, and the result is displayed with supporting information.

2.4 Theoretical Framework

The theoretical foundation of the project is supervised machine learning. A labeled dataset is used to learn a mapping from input features to an output class. Here, the input feature vector consists of attendance, past grades, and study hours, while the target class is At Risk or Safe.

KNN classifies an observation by comparing it with training observations using a distance measure. The Euclidean distance between two feature vectors can be expressed as: $d(x,y) = \sqrt{\sum (x_i - y_i)^2}$. After calculating distances, the algorithm selects the k nearest training observations and assigns the class that receives the majority of votes.

Feature scaling is important because a variable with a larger numerical range can otherwise dominate the distance calculation. Standardization transforms a feature approximately according to $z = (x - \mu) / \sigma$, where μ is the mean and σ is the standard deviation of the feature in the training data.

CHAPTER 3: METHODOLOGY

3.1 System Architecture

The system follows a simple data-to-prediction pipeline. Student records are collected from an institutional dataset or a suitable public dataset. The dataset is inspected for missing values, inconsistent entries, and unsuitable fields. Relevant features are selected, processed, and divided into training and testing data. The training data is used to fit the KNN classifier. The trained model is then connected to the web application.

The web application accepts attendance percentage, past grade information, and study hours as input. The same preprocessing transformation used during training is applied to the new input. The kNN model generates a predicted class and the application displays the result as At Risk or Safe.

The architecture can be represented as: Dataset → Data Cleaning → Feature Selection → Scaling → Train/Test Split → KNN Training → Model Evaluation → Model Persistence → Web Application → Prediction and Visualization.

3.2 Hardware and Software Requirements

Category	Requirement	Purpose
Hardware	Computer/Laptop, 4 GB RAM or higher	Development and model execution
Operating System	Windows / Linux / macOS	Application development
Programming	Python 3.x	Data processing and ML
Libraries	pandas, NumPy, scikit-learn	Data handling and kNN
Visualization	Matplotlib / Seaborn	Charts and exploratory analysis
Web Framework	Flask or Streamlit	Web application interface
Development	VS Code / Jupyter Notebook	Coding and experimentation
Data	CSV dataset	Student academic records

3.3 Dataset and Data Preparation

The project can use institutional academic records or the UCI Student Performance dataset. For the proposed web application, the essential predictive variables are attendance percentage, past grades, and study hours. A binary target variable is required to train the classifier. If the source dataset does not directly contain an At Risk/Safe label, an academically justified labeling rule should be defined using the available outcome variables and documented clearly.

Data preparation includes checking data types, removing or handling missing values, detecting invalid ranges, selecting relevant features, encoding the target label, and scaling numerical features. The preprocessing pipeline must be fitted only on the training data and then applied to validation, test, and new user inputs to avoid data leakage.

A typical split is 80% training data and 20% testing data, although the final split should be selected according to dataset size and evaluation requirements. Stratified splitting is preferable for classification when the class distribution is uneven.

3.4 KNN Model Design

The KNN classifier stores the training observations rather than learning a conventional parametric decision function. During prediction, it calculates distances between the new student record and training records. The value of k controls how many neighbors participate in the voting process.

- Choose a candidate range of k values, such as 3, 5, 7, and 9.
- Scale the numerical input features before distance calculation.
- Train the classifier using the training set.
- Evaluate candidate k values using validation or cross-validation.
- Select a suitable k based on evaluation performance and stability.
- Test the final model only after model selection is complete.

3.5 Web Application Design

The web application is designed around a simple prediction workflow. The home page introduces the purpose of the system. The prediction page provides input fields for attendance, past grades, and study hours. After submission, the backend validates the values, applies the saved preprocessing pipeline, and passes the transformed input to the trained KNN model.

The result page displays the predicted category. A supporting dashboard can show dataset summaries, class distribution, feature relationships, and model evaluation metrics. The interface should use clear labels and avoid presenting the classification as a guaranteed outcome.

3.6 Implementation Process

- Create or obtain the student performance dataset.
- Load the CSV file using pandas and inspect shape, columns, data types, and missing values.
- Clean and preprocess the selected features.
- Create or obtain the binary target label.
- Split the dataset into training and testing sets.
- Standardize numerical features using a training-fitted scaler.
- Train the KNN classifier and tune the k value.
- Evaluate the model using accuracy, precision, recall, F1-score, and confusion matrix.
- Create visualizations for academic patterns and model performance.
- Save the trained model and preprocessing pipeline.
- Integrate the model into a Flask or Streamlit web application.
- Test the complete prediction workflow using valid and boundary input values.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Testing Methodology

Testing is performed at both the machine learning level and the web application level. Model testing uses a held-out test set that is not used for training or k selection. Web application testing checks input validation, preprocessing consistency, prediction generation, result display, and handling of invalid values.

Representative test cases should include a student with strong attendance, strong grades, and adequate study hours; a student with weak indicators; and students with mixed indicators. Boundary values such as 0 and 100 for percentages should also be validated according to the allowed input range.

4.2 Performance Metrics

Accuracy measures the proportion of correctly classified observations. Precision measures how many observations predicted as a particular positive class are actually positive. Recall measures how many actual positive observations are correctly identified. The F1-score provides a balance between precision and recall.

For an academic-risk system, recall for the At Risk class is especially important because failing to identify a genuinely at-risk student may delay support. However, very high recall accompanied by excessive false positives may also reduce the usefulness of the system. Therefore, multiple metrics should be considered together.

The confusion matrix summarizes true positives, true negatives, false positives, and false negatives. It provides a direct view of the types of errors made by the classifier.

4.3 Visualization and Prediction Analysis

The dashboard can include a class-distribution chart, attendance-versus-grade scatter plot, study-hours-versus-grade scatter plot, and a confusion matrix. These visualizations help users understand the data and evaluate whether the model behaves consistently with the observed training patterns.

The following table is an illustrative layout for reporting final model results. The numerical values should be replaced with the actual results obtained after training the final model.

Metric	Illustrative Value	Interpretation
Accuracy	Replace with measured result	Overall proportion of correct predictions
Precision	Replace with measured result	Reliability of positive risk predictions
Recall	Replace with measured result	Ability to identify actual risk cases
F1-score	Replace with measured result	Balance between precision and recall

4.4 Discussion

The expected benefit of the system is not only the classification output but also the integration of prediction with understandable academic indicators. A student record can be examined in the context of attendance, grades, and study habits rather than relying on a single measure.

The effectiveness of KNN depends on the quality of the dataset, feature scaling, the choice of k, and the class-label definition. If the training data is small or contains noisy observations, the prediction may be unstable. Consequently, model results should be validated carefully before any real institutional deployment.

The application should be treated as a decision-support prototype. Educators should review the broader academic and contextual information before taking action based on a prediction.

CHAPTER 5: REFLECTION ON LEARNING

5.1 Technical Skills Acquired

The development of the Student Performance Risk Predictor strengthened my understanding of Python programming, data structures, data preprocessing, exploratory data analysis, visualization, and supervised machine learning. I gained practical experience in loading CSV data, inspecting columns and data types, handling missing values, selecting relevant features, and preparing data for classification.

I also developed a clearer understanding of the KNN algorithm, Euclidean distance, nearest-neighbor voting, feature scaling, training and testing, and model evaluation. Integrating the trained model into a web application provided experience in connecting machine learning logic with a user-facing interface.

5.2 Data Handling and Visualization Skills

The project demonstrated the importance of data quality before model training. I learned that incorrect data types, missing values, inconsistent ranges, and unsuitable labels can affect both visualization and prediction quality.

Creating visualizations helped me understand relationships between attendance, grades, and study hours. Scatter plots can reveal relationships between numerical variables, while bar charts can summarize the number of students in each risk category. These visualizations make data easier to interpret and communicate.

5.3 Problem Solving and Communication

The project required breaking a broad educational problem into smaller technical tasks: data acquisition, preprocessing, model selection, evaluation, web integration, and testing. This improved my ability to identify dependencies between stages and troubleshoot problems systematically.

Presenting the prediction in simple language also improved my communication skills. A machine learning model is useful only when users can understand what its output means and what its limitations are.

5.4 Application of Engineering Practices

The project followed a structured development approach involving requirements analysis, design, implementation, testing, and documentation. Maintaining a clear separation between preprocessing, model training, prediction, and interface components improves maintainability.

Data privacy is also important in an educational context. Institutional records should be handled securely, access should be restricted to authorized users, and personally identifying information should not be exposed unnecessarily.

5.5 Personal Development

This project improved my confidence in applying theoretical data science concepts to a practical problem. It helped me understand that building a useful machine learning application involves more than selecting an algorithm; it also requires careful data preparation, evaluation, interface design, testing, and responsible interpretation.

Overall, the project strengthened my interest in data-driven applications and provided practical experience that can be applied to future machine learning and web development projects.

CHAPTER 6: CONCLUSION

6.1 Summary of Achievements

The Student Performance Risk Predictor provides a complete academic prototype for classifying students into At Risk or Safe categories using attendance, past grades, and study hours. The project integrates the major stages of the data science workflow: data acquisition, preprocessing, exploratory visualization, supervised learning, model evaluation, and web-based presentation.

The use of KNN provides an intuitive classification mechanism based on similarity between student records. The web interface makes the prediction workflow accessible without requiring users to interact directly with Python code.

6.2 Key Findings

- Academic indicators can be organized into a structured feature set for machine learning classification.
- Feature scaling is important for distance-based algorithms such as KNN.
- The value of k has a direct effect on prediction behavior and should be selected using evaluation rather than assumption.
- Accuracy alone is not sufficient for evaluating an academic-risk classifier; recall, precision, F1-score, and the confusion matrix should also be considered.
- Visualization helps users understand academic patterns and supports interpretation of model outputs.
- The prediction should be treated as an early-warning indicator rather than a final decision about a student.

6.3 Future Scope

- Add more academic and engagement features such as assignment completion, internal assessment scores, previous-semester performance, and learning activity.
- Compare KNN with decision trees, random forests, logistic regression, and other suitable classifiers.
- Use cross-validation and automated hyperparameter tuning for more reliable model selection.
- Add personalized dashboards for students, teachers, and administrators with role-based access.
- Provide trend analysis over multiple semesters rather than relying on a single snapshot.
- Add explainable prediction information showing which academic indicators are associated with the model's decision.
- Deploy the application securely using an institutional server or cloud platform after privacy and governance requirements are satisfied.

6.4 Recommendations

The system should be validated on representative institutional data before any operational use. Class labels and risk thresholds should be defined with appropriate academic stakeholders. Predictions should be reviewed by teachers or authorized staff and should not be used as the sole basis for disciplinary, grading, or high-impact decisions.

For future versions, the project should maintain an auditable preprocessing pipeline, record model versions, monitor performance over time, and periodically retrain the model when the underlying student population or academic environment changes.

CHAPTER 7: REFERENCES

1. Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques. Morgan Kaufmann.
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). The Elements of Statistical Learning. Springer.
3. Géron, A. (2022). Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media.
4. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825–2830.
5. UCI Machine Learning Repository. Student Performance dataset. University of California, Irvine.
6. McKinney, W. (2022). Python for Data Analysis. O'Reilly Media.
7. Grus, J. (2019). Data Science from Scratch. O'Reilly Media.
8. Raschka, S., & Mirjalili, V. (2019). Python Machine Learning. Packt.
9. Waskom, M. (2021). seaborn: statistical data visualization. Journal of Open Source Software.
10. Matplotlib Development Team. Matplotlib Documentation.